

1. [5] Considere a classe **UnsafeSuccession**, que tem por objectivo representar uma sequência de elementos, consumidos de forma sequencial através do método **next**. Assuma que os elementos do *array* não são alterados após a construção da instância da sequência.

```
class UnsafeSuccession<T>(  
    private val items: Array<T>  
) {  
    private var index = 0  
    fun next(): T? =  
        if(index < items.size) {  
            items[index++]  
        } else {  
            null  
        }  
}
```

- a. [2] Realize uma versão *thread-safe* desta classe, com recurso ao uso de *locks*.
- b. [3] Realize uma versão *thread-safe* desta classe, sem recurso ao uso de *locks*.
2. [4] Realize o sincronizador **MessageBroadcaster** com a interface apresentada em seguida.

```
class MessageBroadcaster<T> {  
    @Throws(InterruptedException::class)  
    fun waitForMessage(timeout: Duration): T? { ... }  
    fun sendToAll(message: T): List<Thread> { ... }  
}
```

A função **waitForMessage** bloqueia a *thread* onde foi realizada a invocação até que uma mensagem seja enviada através da função **sendToAll**, ou até que tenha decorrido o tempo definido em **timeout**. No primeiro caso é retornada a mensagem enviada, no segundo caso é retornado **null**. A função **waitForMessage** deve ser sensível à interrupção da *thread* onde foi chamada. A função **sendToAll** deve retornar a lista com as *threads* que receberam a mensagem, isto é, em que a chamada a **waitForMessage** retornou a mensagem entregue. A lista retornada pode estar vazia (não existiam *threads* à espera de mensagem), ou não (existia pelo menos uma *thread* à espera de mensagem). A mensagem passada na chamada **sendToAll** não deve ficar disponível para chamadas futuras da função **waitForMessage**.

3. [4] Realize o sincronizador **ThreadPool** com a interface apresentada em seguida

```
class ThreadPool(  
    nOfThreads: Int  
) {  
    @Throws(RejectedExecutionException::class)  
    fun execute(runnable: Runnable): Unit { ... }  
    @Throws(InterruptedException::class)  
    fun shutdownAndWaitForTermination(): Unit { ... }  
}
```

Este *thread pool* tem um número fixo de *threads*, criadas aquando da construção do *pool* e que terminam apenas quando o *pool* for encerrado (*shutdown*). As *threads* que pretendem executar funções através do *thread pool* invocam o método **execute**, especificando o item de trabalho a executar com o argumento **runnable**. Este método retorna imediatamente. A chamada ao método **shutdownAndWaitForTermination** coloca o executor em modo de encerramento. Neste modo, todas as chamadas ao método **execute** deverão lançar a exceção **RejectedExecutionException**. Contudo, todas as submissões para execução feitas antes da chamada ao método **shutdownAndWaitForTermination** devem ser processadas normalmente. A chamada a este método retorna normalmente apenas após a conclusão do processo de encerramento do executor, isto é, aguarda até que sejam executados todos os itens de trabalho aceites e que todas as *worker threads* terminem. A função **shutdownAndWaitForTermination** deve ser sensível à interrupção da *thread* onde foi chamada, contudo o processo de *shutdown* é irreversível.

4. [4] Realize a classe **SynchronousQueue<T>** para comunicação de mensagens entre *coroutines*, através de filas de mensagens *first in first out*, disponibilizando os métodos:

- a. **suspend fun enqueue(message: T): Unit**
- b. **suspend fun dequeue(): T**

Ambos os métodos **não** devem suportar cancelamento. A capacidade interna da fila é zero, ou seja, uma operação de *enqueue* só termina quando a mensagem for entregue diretamente a uma operação de *dequeue*.

5. [3] Realize a função
- | | | | | | | |
|---------|-----|-----|----|----|----------------------|-----------------------|
| suspend | fun | <T, | U, | R> | combineFutures(f1: | CompletableFuture<T>, |
| | | | | | f2: | CompletableFuture<U>, |
| | | | | | combiner: (T,U)->R): | R |

Esta função retorna o resultado da aplicação da função **combiner** aos valores com que os *futures* **f1** e **f2** se completarem. Caso um dos *futures* se complete com exceção, a função **combineFutures** deve lançar essa exceção. A implementação da função **combineFutures** não deve recorrer ao bloqueio de *threads*, exceto para aquisição de exclusão mútua.

Duração: 3 horas
ISEL, 30 de junho de 2023